

Enhancing HPX with FleCSI: Automatic Detection of Implicit Task Dependencies

S. Davis Herring, Maxim Moraru, **Scott Pakin**, Julien
Loiseau, Richard Berger, Philipp V. F. Edelmann, and
Ben Bergen

16 November 2025

LA-UR-25-31117

Outline

- Background
- Implementation
- Analysis
- Conclusions

Motivation: Managing Complexity

HPC hardware complexity

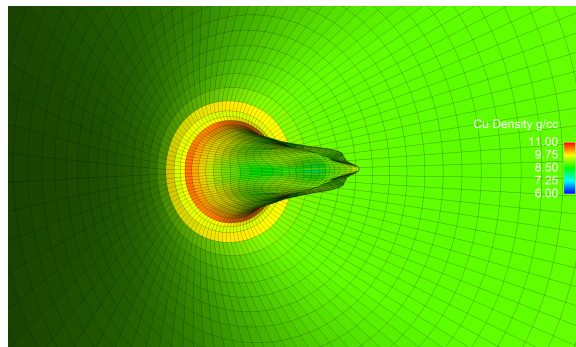
- Large numbers of nodes
- Many CPU cores and multiple GPUs per node
- Numerous system-wide address spaces and NUMA domains
- Various communication mechanisms
 - Shared memory, message passing
- Exact configurations differ across supercomputers

HPC application complexity

- Multiple interacting physics methods
- Irregular data structures
- Various forms of parallelism
 - Task parallelism, data parallelism



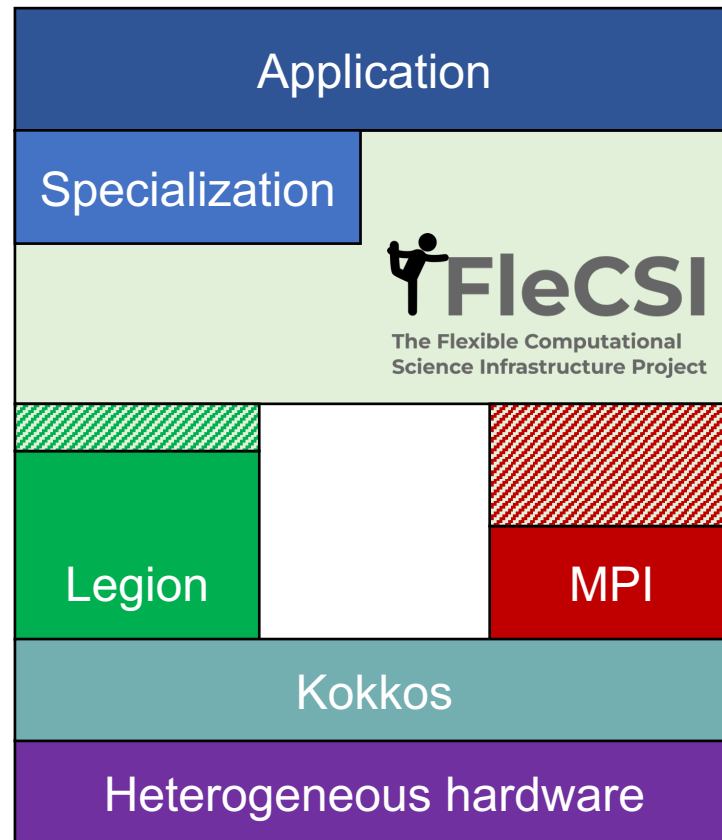
LANL's forthcoming Mission supercomputer



Jet formed from a shaped-charge simulation, Lagrangian hydrodynamics

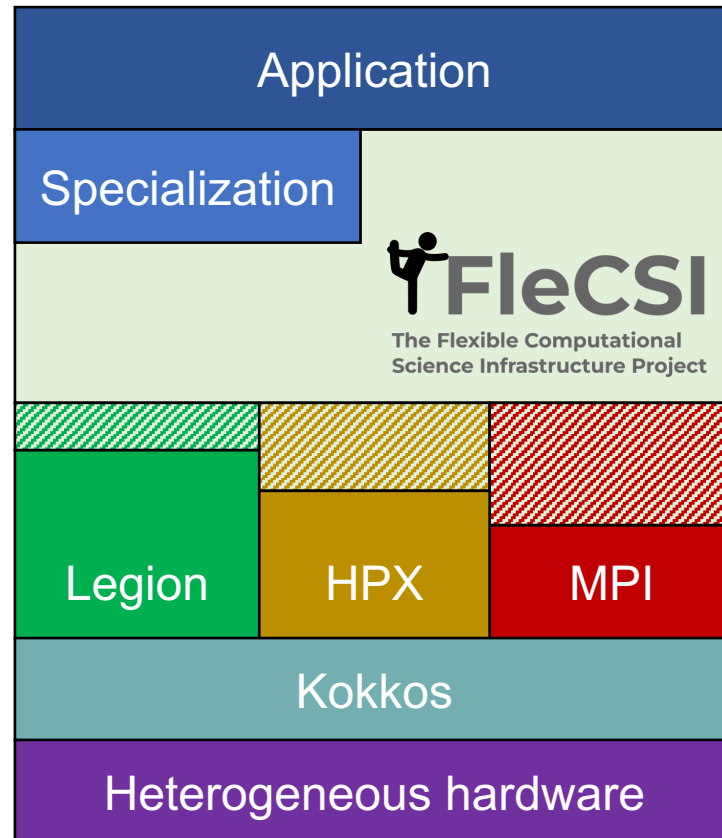
The FleCSI Programming Model

- Data are distributed globally
 - “Topologies” provide commonly needed, generic distributed data structures
 - “Specializations” aggregate and customize topologies for a specific application
- Tasks declare the data they’ll read & write
 - Divided among multiple instances (“point tasks”)
- FleCSI automatically parallelizes tasks subject to data dependencies
- Communication is implicit
 - Before a task begins, its data are made available locally



This Work

- FleCSI supports different backends
 - Legion: similar data and execution models to FleCSI's; minimal interfacing required
 - MPI: communication only; tasking and dependency management must be implemented on top
- Intention is for applications to run unmodified on different backends
- What would it take to implement a new HPX backend for FleCSI?
 - HPX: provides tasking and DAG structure; data model must be implemented on top



Contrasting the FleCSI and HPX Models

FleCSI	HPX
Task launches spawn multiple, concurrent point tasks	Only point tasks exist
Task ordering is based on data dependencies	Task ordering is based on explicitly specified task dependencies
No concept of communicators	Communicator objects needed for broadcasting and reducing task results

→ Goal is to implement the FleCSI model in terms of the HPX API

Outline

- Background
- **Implementation**
- Analysis
- Conclusions

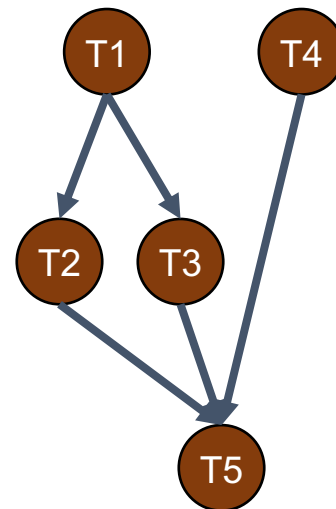
Dependency Inference

- FleCSI fields
 - Divided into *exclusive*, *shared*, and *ghost* subsets
 - Access rights can be expressed as *read* and/or *write*
- FleCSI tasks are parallelized based on access rights
 - A field can be read by multiple tasks at once
 - Writes to a field serialize accesses
- HPX tasks specify explicit dependencies
 - A task launch returns a *future*
 - Other tasks can request to block on that future's completion
- Implementing FleCSI's parallelization model
 - Maintain a frontier of HPX futures
 - For each field, for each subset, track the most recent (single) writer and (multiple) readers

Example of Dependency Inference

Time ↓

Task	Field accesses	Most recent A		Most recent B	
		Writer	Readers	Writer	Readers
T1	Write A	T1			
T2	Read A	T1	T2		
T3	Read A	T1	T2, T3		
T4	Write B	T1	T2, T3	T4	
T5	Write A & B	T5		T5	



Communicator Reuse

- HPX communicators, used by broadcasts and reductions of task results, are expensive to create
- Want to aggressively recycle existing communicators
- Challenge
 - HPX tasks execute asynchronously
 - How can we determine when *no* active task is holding a given communicator?
- Insight
 - Planning is separate from execution—a communicator can be *selected* for use by multiple tasks before any of them run
 - Rely on task dependencies to prevent concurrent use by unrelated collective operations



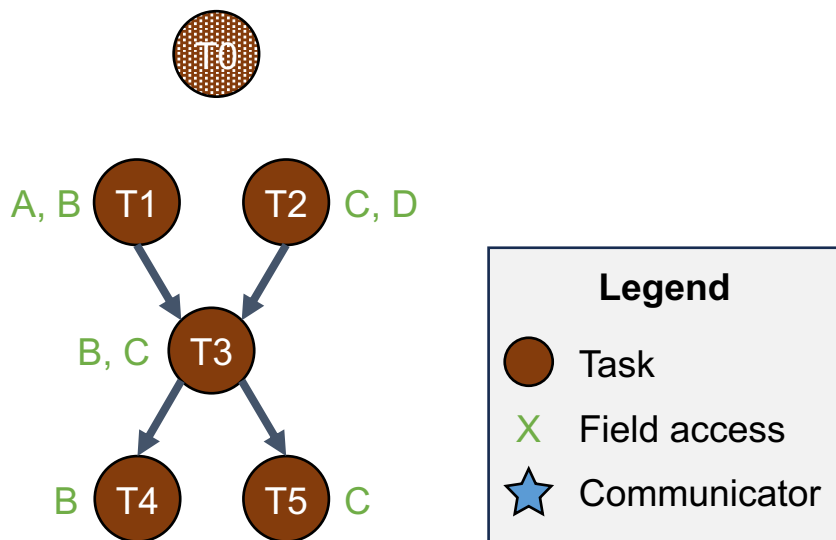
Communicator Reuse (cont.)

- Maintain a separate graph from the HPX task graph
 - Nodes still represent tasks
 - A node points to nodes from which it *can* inherit communicators
- Contracted aggressively to reduce graph-traversal costs



Example of Communicator Reuse

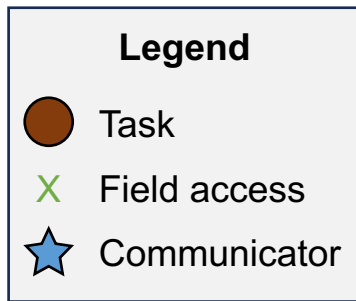
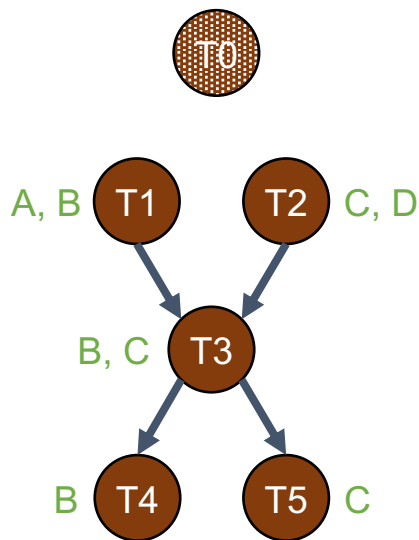
- Consider the following task graph:



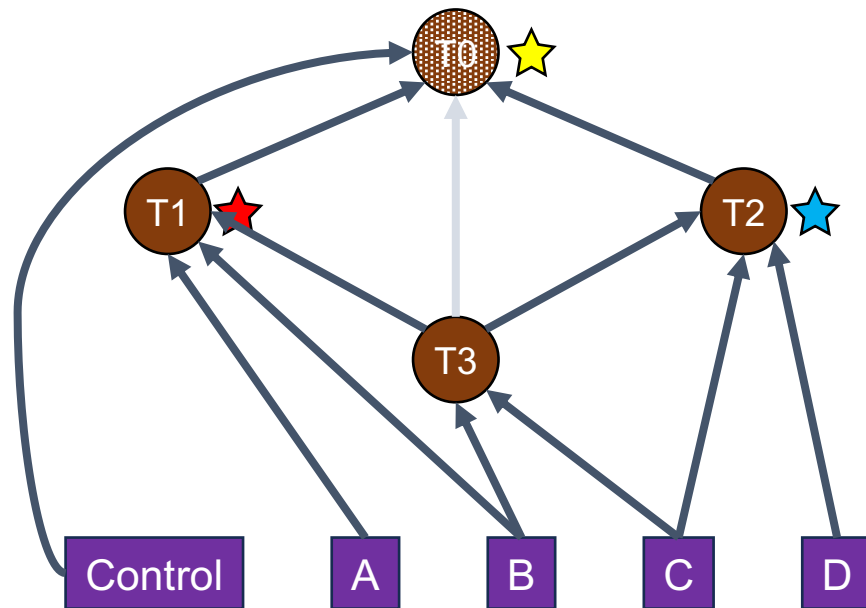
- Assumptions
 - Task T0 was waited on before the other tasks were launched
 - All tasks except task T3 require a communicator
- Intuition behind the communicator-reuse algorithm
 - T1 & T2 and T4 & T5 need different communicators
 - Communicator transfers should flow through T3, which doesn't need a communicator
 - T4 and T5 can reuse T1's and T2's communicators

Example of Communicator Reuse

- Consider the following task graph:

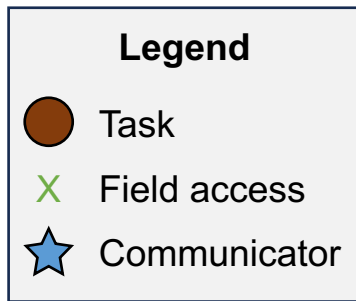
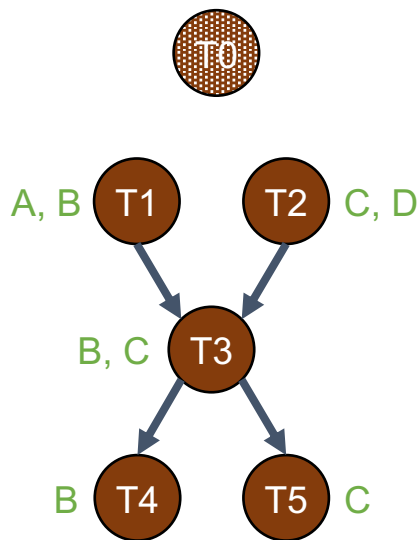


- Communicator propagation:

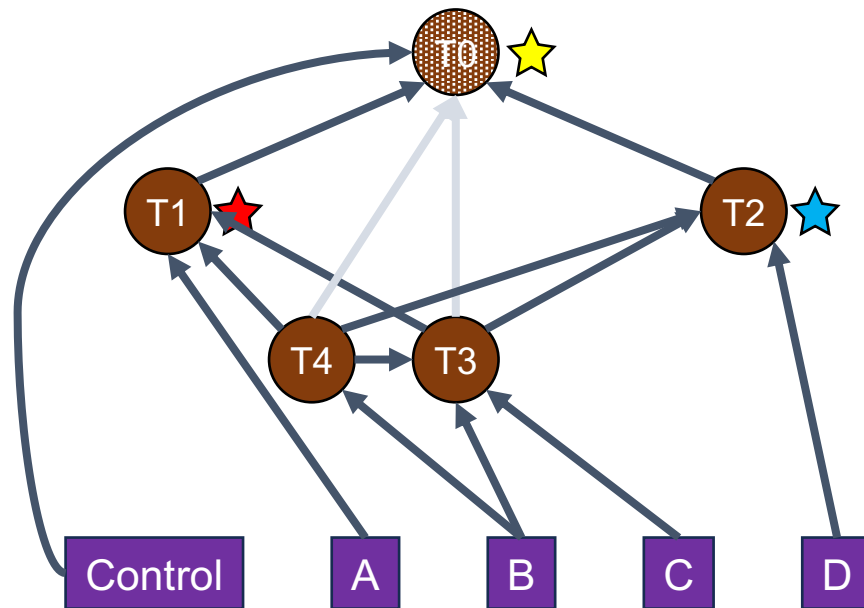


Example of Communicator Reuse

- Consider the following task graph:

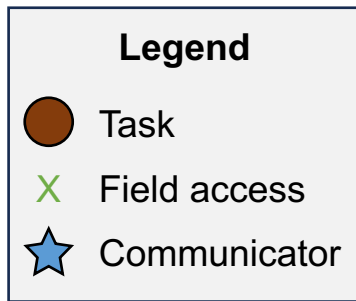
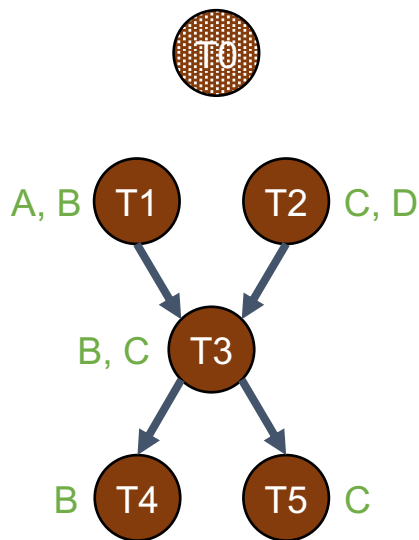


- Communicator propagation:

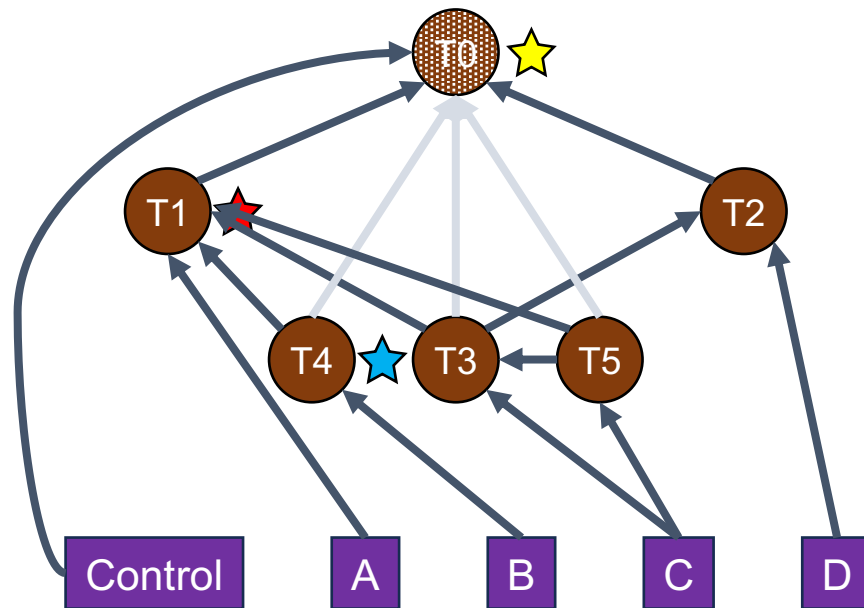


Example of Communicator Reuse

- Consider the following task graph:



- Communicator propagation:



Outline

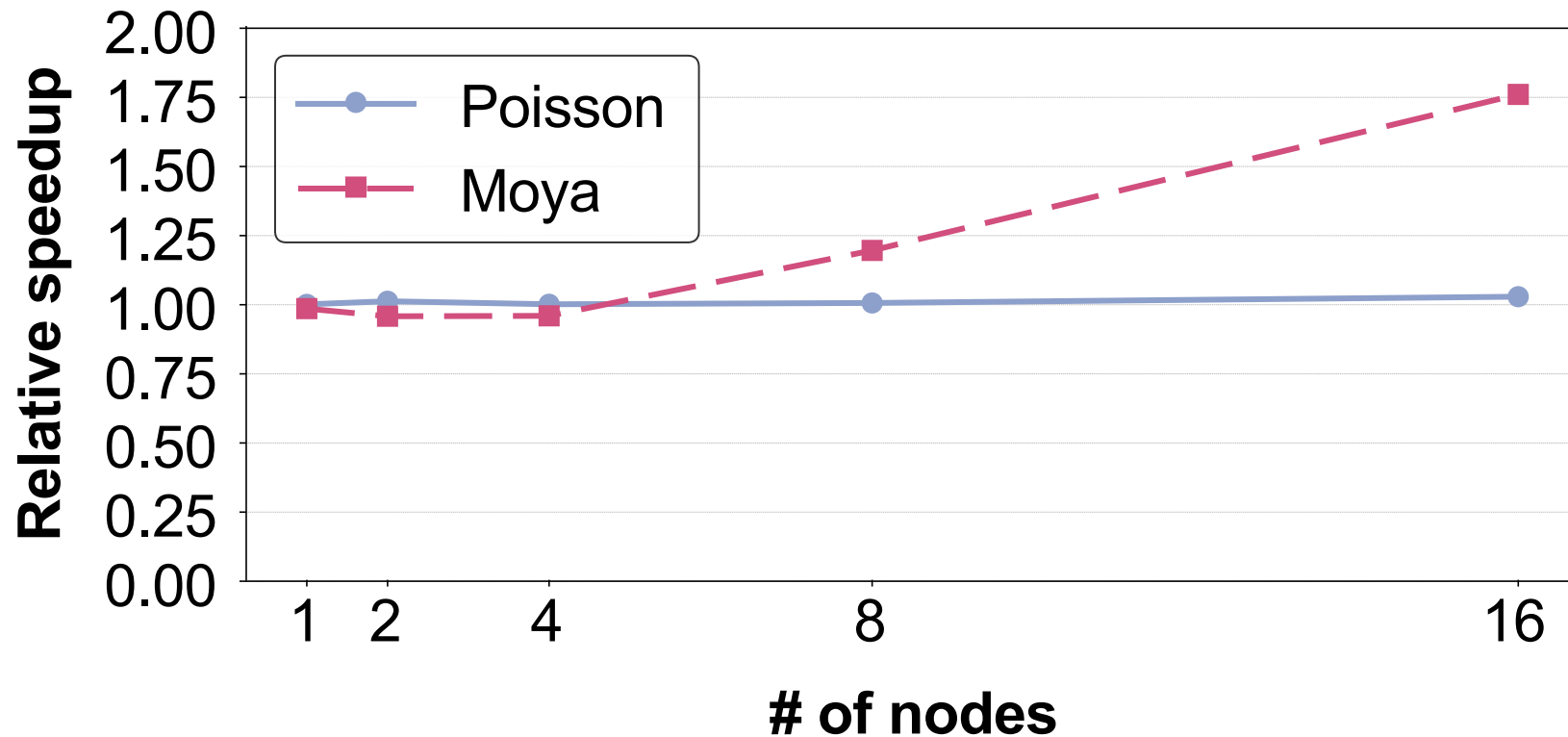
- Background
- Implementation
- **Analysis**
- Conclusions

Does Communicator Reuse Improve Performance?

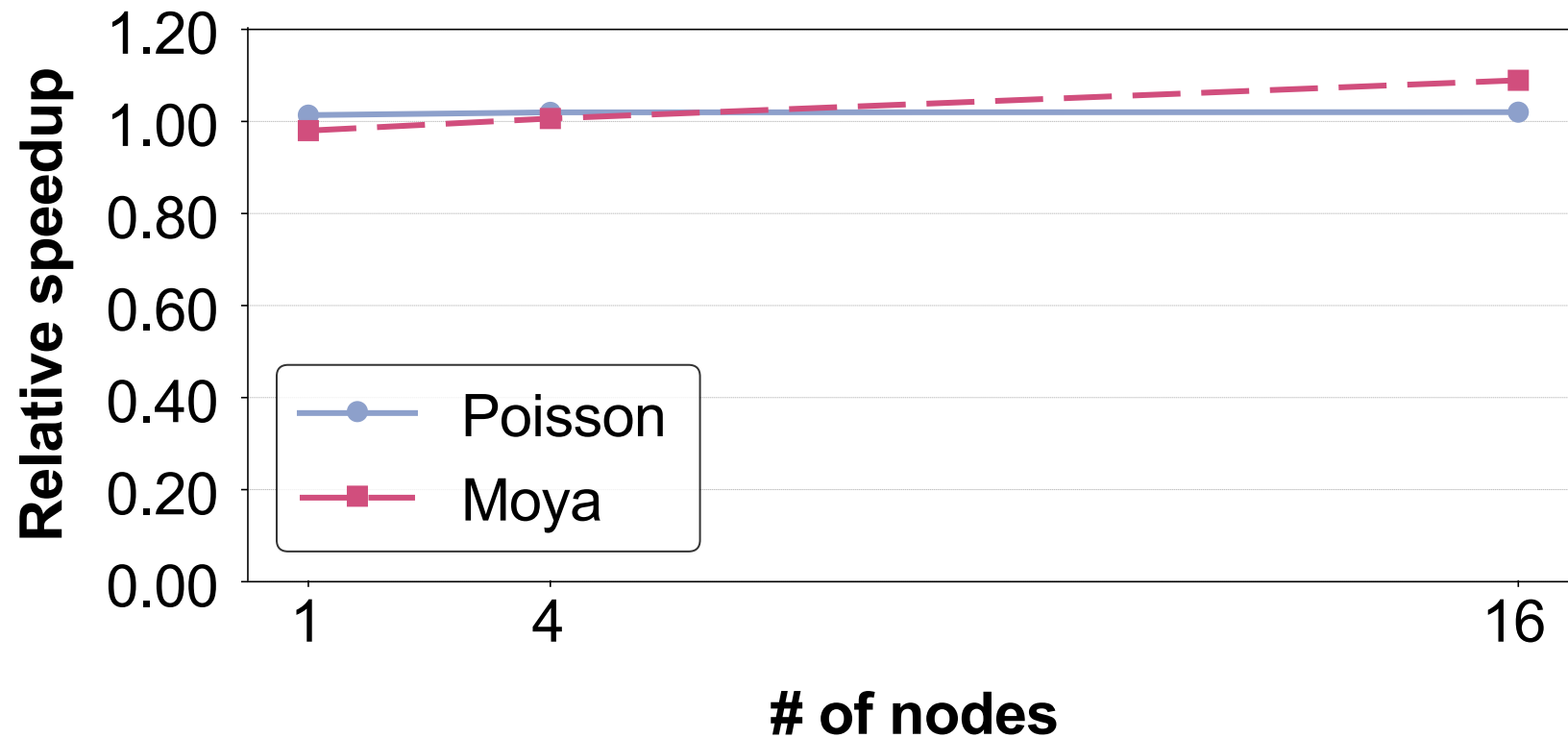
- Test applications
 - **Poisson**: toy code that solves the two-dimensional Poisson equation on a structured mesh with red-black Gauss–Seidel (5-point stencil)
 - **Moya**: low-energy-density multiphysics simulation in arbitrary Lagrangian–Eulerian frames (unstructured mesh deforming with flow or fixed mesh that flow passes through)
- Observed communicator counts

Application	Communicators allocated during initialization		Communicators allocated during execution		
	Without reuse	With reuse	Without reuse	With reuse	
Poisson	4	1	2	0	} Per 100 iterations
Moya	67	4	7	0	} Per timestep

Strong-Scaling Results



Weak-Scaling Results



Outline

- Background
- Implementation
- Analysis
- Conclusions

Conclusions

- Reusing communicators can improve performance
 - Especially at larger scales
 - Especially in latency-sensitive applications
- FleCSI can target significantly different backends (Legion, MPI, and now HPX) without requiring application modifications or performance hints
- Careful management of communication setup costs can play an important role in sustaining scalability in task-based run-time systems